Employee Management System

Technical Architecture & Design Document

Full-Stack RBAC Application

Spring Boot 3.2 + React 18 + MySQL 8.0

Version 1.0 | February 2026

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Entity Relationship Diagram
5. Authentication & JWT Flow
6. Role-Based Access Control
7. Employee CRUD Operations
8. Daily Task Submission Flow
9. Frontend-Backend Communication
10. API Endpoints Reference
11. Database Schema
12. Security Architecture
13. Frontend Pages & Components
14. Deployment Configuration
15. Demo Credentials

1. Project Overview

The **Employee Management System (EMS)** is a full-stack web application that provides comprehensive employee management capabilities with role-based access control. The system supports three user roles — **ADMIN**, **HR**, and **EMPLOYEE** — each with distinct permissions and UI experiences.

Key Features

Feature	Description
Authentication	JWT-based with dynamic per-user signing secrets
Employee Management	Full CRUD operations with role-gated access
Department Management	Department organization with employee count tracking
Daily Task Tracking	Department-specific daily task checklists with submission history
Self-Service Profile	Employees can view/edit their own profile and change passwords
Role-Based UI	Dynamic navigation and page access based on user role

Business Rules

- **Auto User Creation**: When an employee is created by ADMIN/HR, a User account (role: EMPLOYEE, default password: `password123`) is automatically created and linked.
- **Salary Protection**: Only ADMIN/HR can modify employee salary fields.
- **One Submission Per Day**: Each employee can submit their daily task completion report only once per day.
- **Password Change**: Changing password invalidates the JWT token, forcing re-authentication.
- **Dynamic JWT Secrets**: Each login generates a fresh 256-bit signing key per user, ensuring token invalidation on logout.

2. Technology Stack

Backend

Technology	Version	Purpose
Java	17	Programming Language
Spring Boot	3.2.0	Application Framework
Spring Security	6.x	Authentication & Authorization
Spring Data JPA	3.x	Data Access Layer (Hibernate ORM)
JJWT	0.12.3	JWT Token Generation & Validation
MySQL	8.0	Relational Database
Maven	3.x	Build & Dependency Management
BCrypt	—	Password Hashing

Frontend

Technology	Version	Purpose
React	18.x	UI Library
Vite	5.x	Build Tool & Dev Server

Material UI (MUI)	5.x	Component Library
React Router	6.x	Client-Side Routing
Axios	1.x	HTTP Client
jwt-decode	4.x	JWT Token Decoding

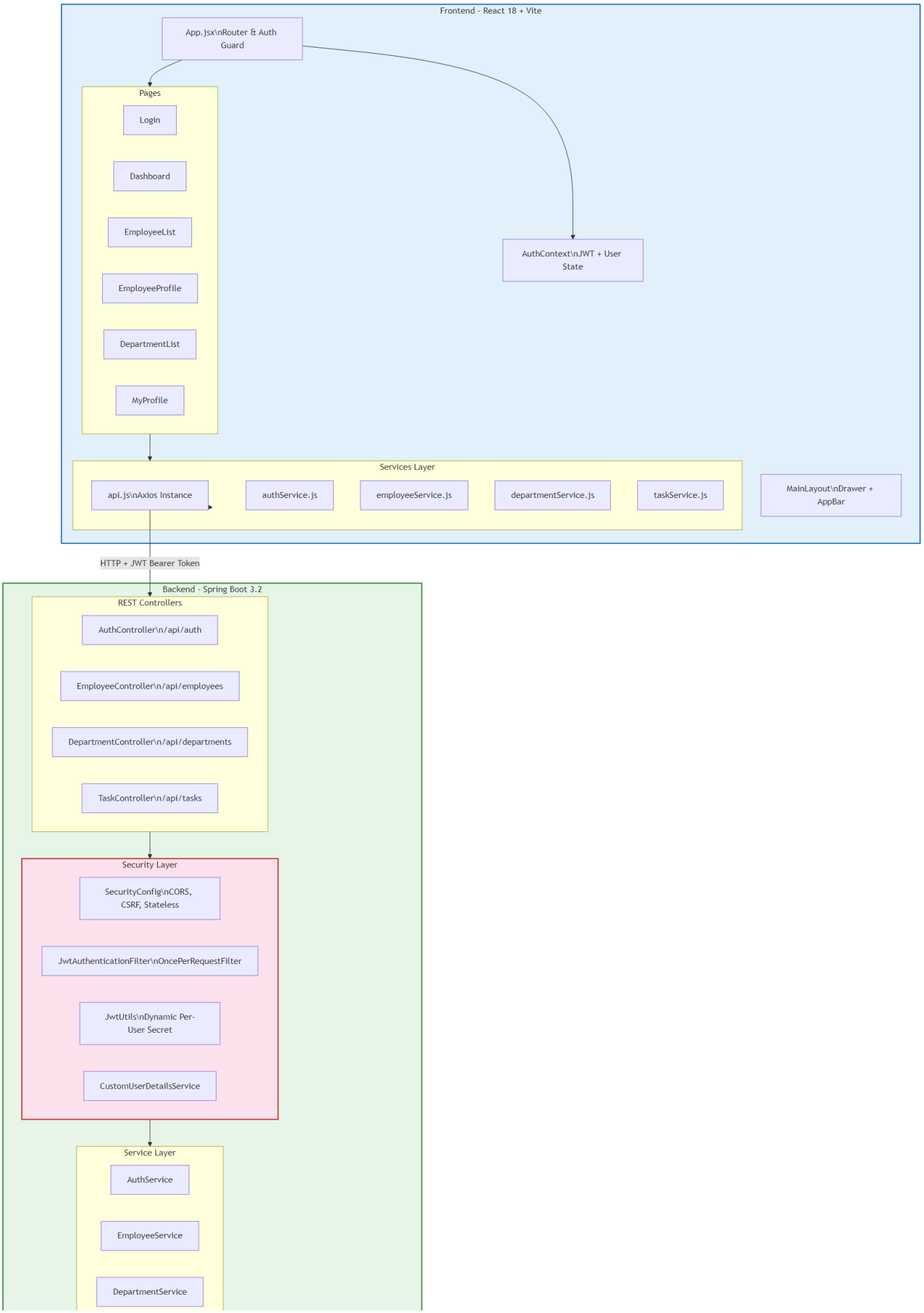
Infrastructure

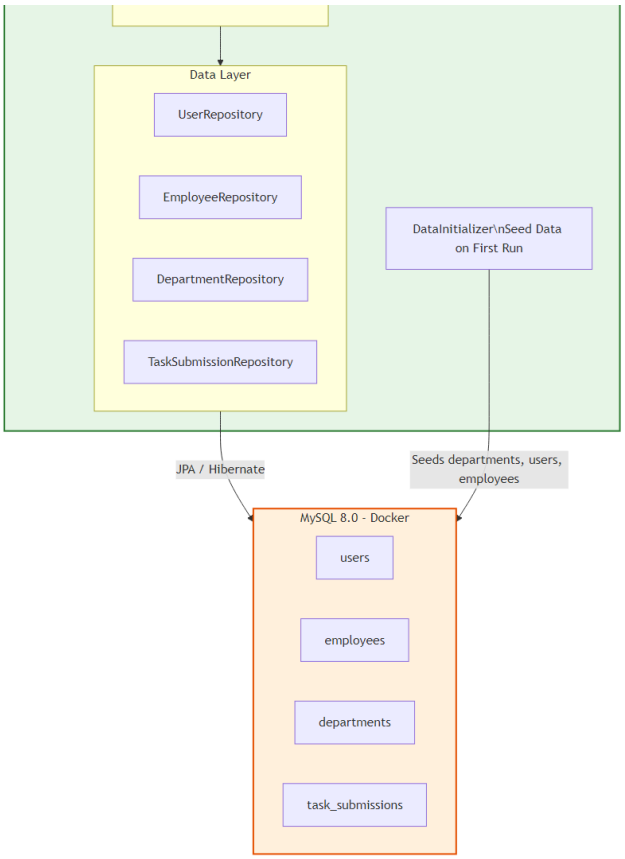
Technology	Purpose
Docker Compose	MySQL container orchestration
Vite Proxy	API proxying from :5173 → :8080

3. System Architecture

The application follows a classical **3-tier architecture** with clear separation of concerns:

- **Presentation Layer**: React SPA with MUI components, AuthContext for state management
- **Application Layer**: Spring Boot REST controllers, service layer with business logic, JWT security filter chain
- **Data Layer**: JPA repositories with Hibernate ORM accessing MySQL database


Frontend - React 18 + Vite
App.jsx\nRouter & Auth Guard
Pages
Login
Dashboard
EmployeeList
EmployeeProfile
DepartmentList
MyProfile
AuthContext\nJWT + User State
Services Layer
api.js\nAxios Instance
authService.js
employeeService.js
departmentService.js
taskService.js
MainLayout\nDrawer + AppBar
HTTP + JWT Bearer Token
Backend - Spring Boot 3.2
REST Controllers
AuthController\n/api/auth
EmployeeController\n/api/employees
DepartmentController\n/api/departments
TaskController\n/api/tasks
Security Layer
SecurityConfig\nCORS, CSRF, Stateless
JwtAuthenticationFilter\nOncePerRequestFilter
JwtUtils\nDynamic Per-User Secret
CustomUserDetailsService
Service Layer
AuthService
EmployeeService
DepartmentService

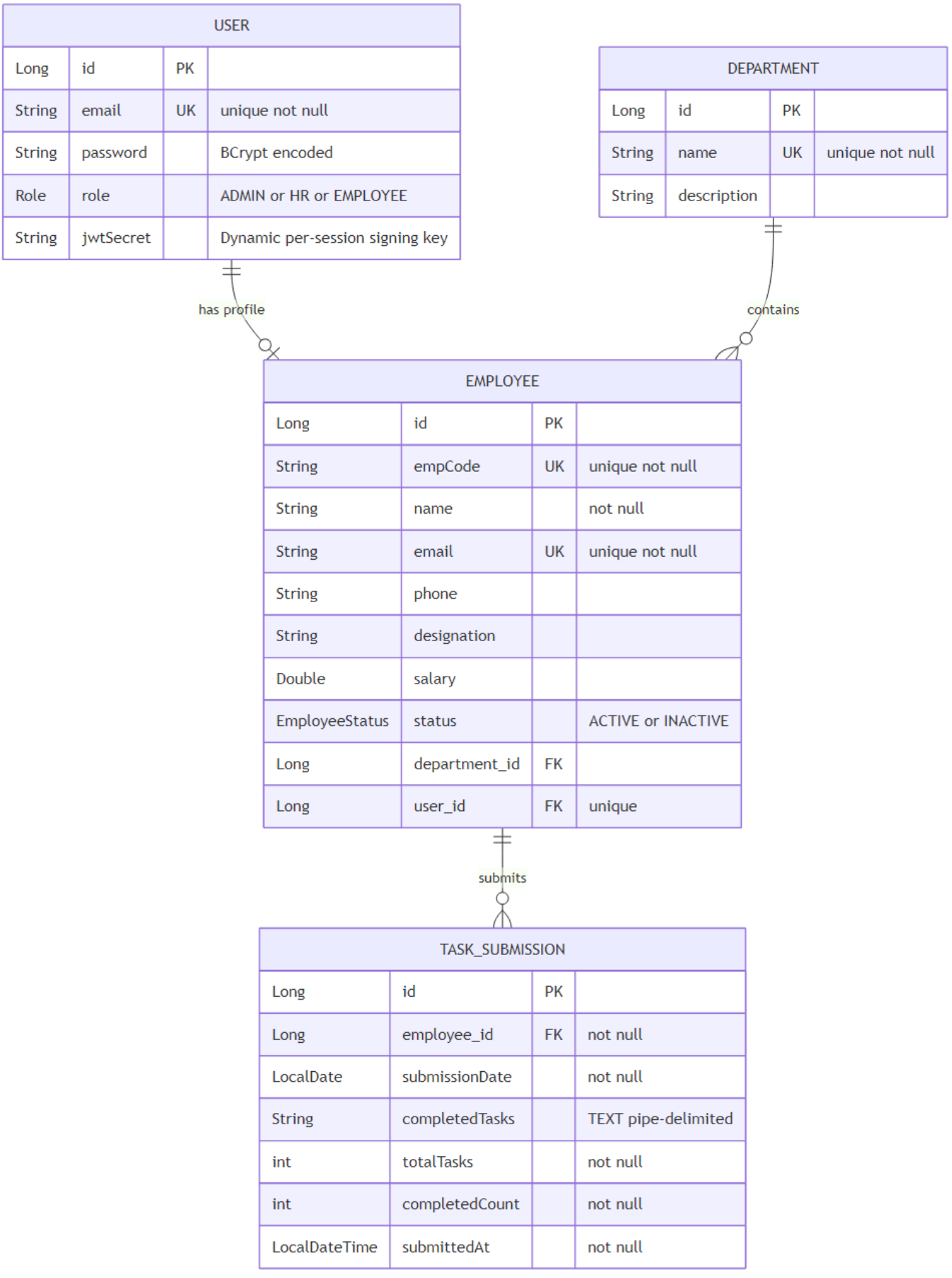


Architecture Highlights:

- Stateless REST API with JWT authentication
- Per-user dynamic JWT signing secrets (not a static application secret)
- Vite dev server proxies `/api` requests to Spring Boot backend
- Spring Security filter chain intercepts every request for authentication
- DataInitializer seeds initial departments, users, and employees on first boot

4. Entity Relationship Diagram

The database consists of **4 core entities** with well-defined relationships:


USER
Long id PK
String email UK unique not null
String password BCrypt encoded
Role role ADMIN or HR or EMPLOYEE
String jwtSecret Dynamic per-session signing key
DEPARTMENT
Long id PK
String name UK unique not null
String description
has profile
contains
EMPLOYEE
Long id PK
String empCode UK unique not null
String name not null
String email UK unique not null
String phone
String designation
Double salary
EmployeeStatus status ACTIVE or INACTIVE
Long department_id FK
Long user_id FK unique
submits
TASK_SUBMISSION
Long id PK
Long employee_id FK not null
LocalDate submissionDate not null
String completedTasks TEXT pipe-delimited
int totalTasks not null
int completedCount not null
LocalDateTime submittedAt not null

Relationships

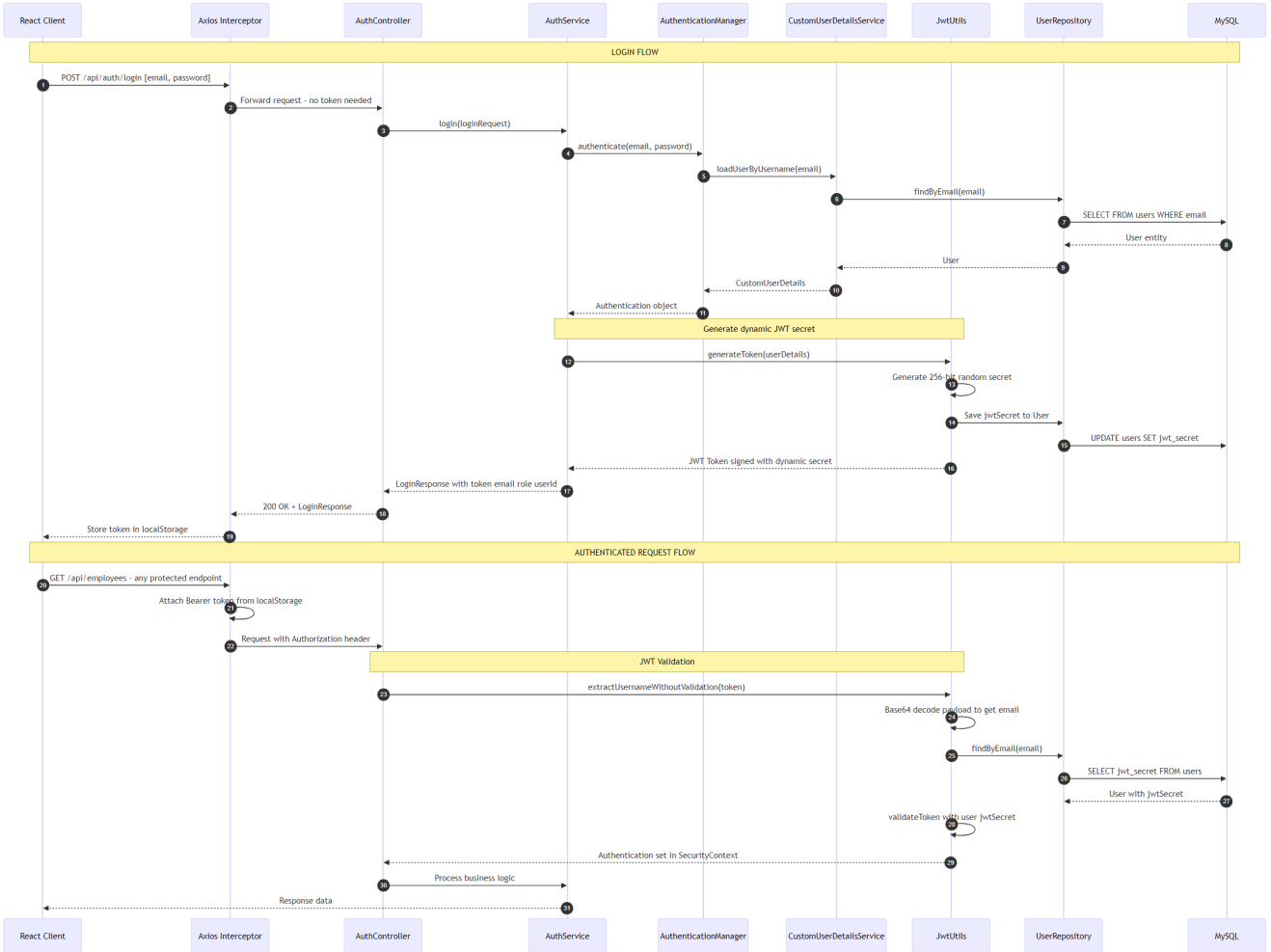
Relationship	Type	Description
User → Employee	One-to-One	Each User optionally has one Employee profile (ADMIN may not have an employee record)
Department → Employee	One-to-Many	A department contains multiple employees
Employee → TaskSubmission	One-to-Many	An employee can have multiple daily task submissions

Constraints

- `User.email` is unique — serves as the login identifier
- `Employee.empCode` is unique — business identifier for each employee
- `Employee.email` is unique — linked to User.email
- `TaskSubmission` has a unique constraint on `(employee_id, submissionDate)` — one submission per employee per day
- `Employee.user_id` is unique — enforces 1:1 with User

5. Authentication & JWT Flow

The system implements a **dynamic per-user JWT secret** pattern for enhanced security:



How It Works

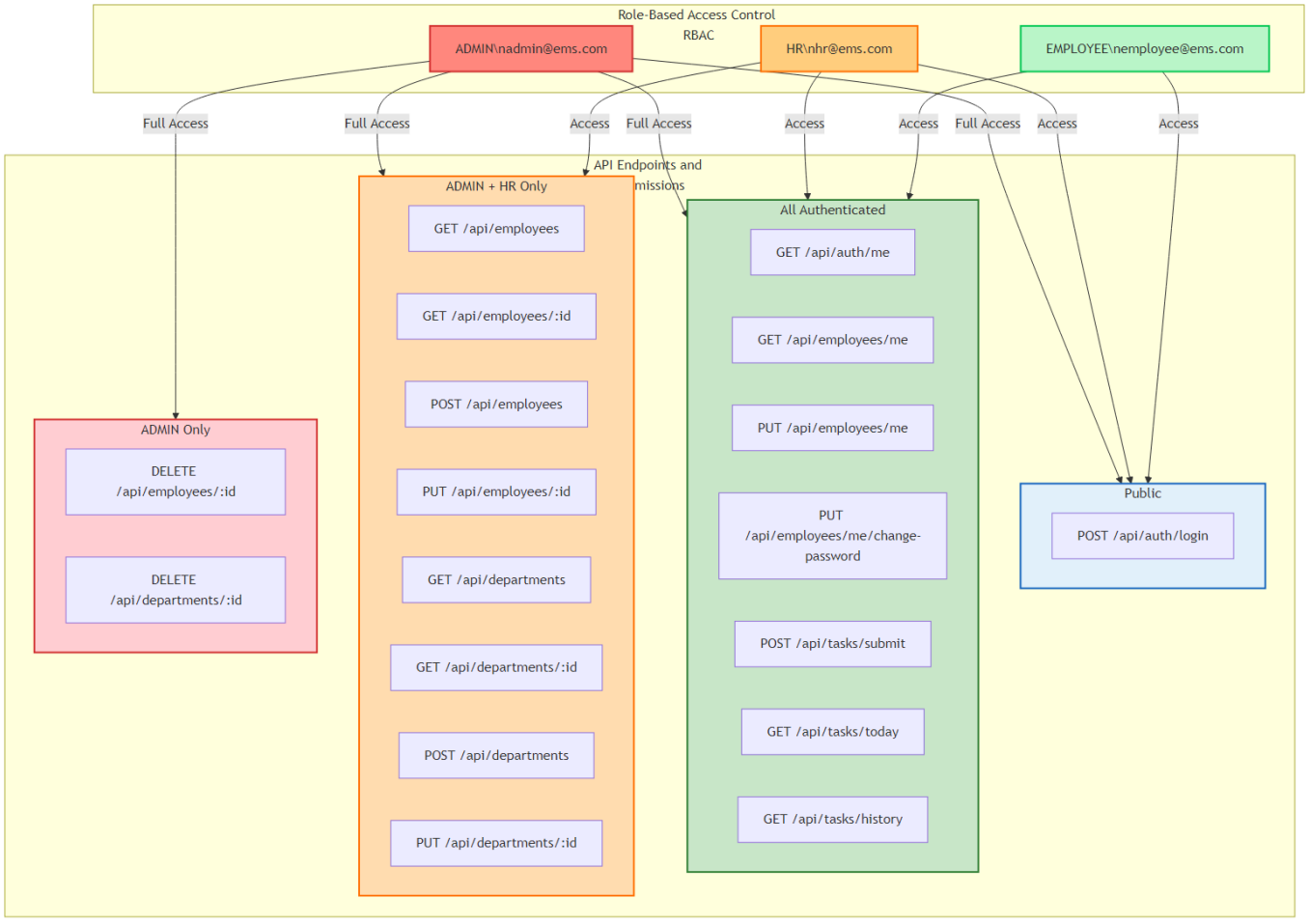
1. **Login**: User submits credentials → Spring's `AuthenticationManager` validates via `CustomUserDetailsService` → On success, a fresh 256-bit random secret is generated and stored in `User.jwtSecret` → JWT is signed with this dynamic secret
2. **Subsequent Requests**: Axios interceptor attaches `Bearer <token>` → `JwtAuthenticationFilter` extracts email from JWT payload (Base64 decode, no validation) → Looks up user in DB to get their stored `jwtSecret` → Validates the token with the user-specific secret
3. **Logout/Password Change**: The `jwtSecret` field is cleared → Any existing token becomes unverifiable → Forces re-login

Token Structure

Claim	Source	Value
`sub`	User.email	Email address (primary identifier)
`email`	User.email	Duplicate for easy access
`role`	User.role	ADMIN, HR, or EMPLOYEE
`iat`	System	Issued-at timestamp
`exp`	Config	24 hours from issuance

6. Role-Based Access Control

The system implements RBAC at both the **URL level** (Spring Security config) and **method level** (`@PreAuthorize` annotations):



Permission Summary

Capability	ADMIN	HR	EMPLOYEE
Login	✅	✅	✅
View own profile	✅	✅	✅
Edit own profile (name, phone)	✅	✅	✅
Change own password	✅	✅	✅
Submit daily tasks	✅	✅	✅
View task history	✅	✅	✅
List all employees	✅	✅	❌
Create employees	✅	✅	❌
Update employees	✅	✅	❌
Delete employees	✅	❌	❌
Manage departments	✅	✅	❌
Delete departments	✅	❌	❌

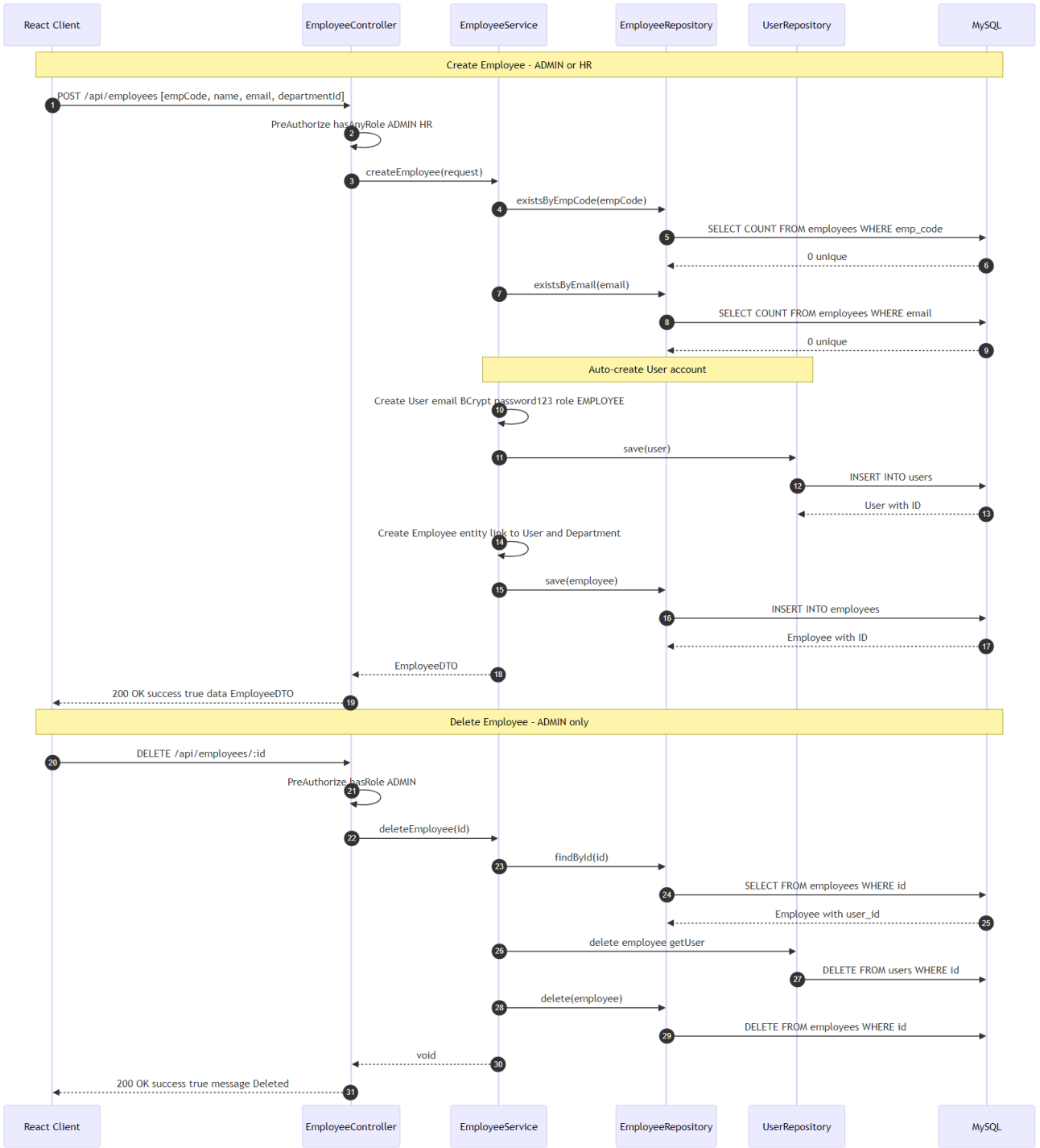
Frontend Role Gating

The `MainLayout` component dynamically shows/hides navigation items based on role:

- **ADMIN/HR**: Dashboard (stats), Employees, Departments
- **EMPLOYEE**: Dashboard (tasks), My Profile

7. Employee CRUD Operations

Employee management is the core feature of the system, with automatic User account creation:



Create Employee Flow

1. ADMIN/HR fills out the employee form (empCode, name, email, department, designation, salary)
2. Backend validates uniqueness of `empCode` and `email`
3. A `User` account is **automatically created** with:
   - Email: same as employee email
   - Password: BCrypt-encoded `password123` (default)
   - Role: `EMPLOYEE`
4. Employee entity is created and linked to both the User and Department
5. Frontend displays the default password notification to the creator

Update Employee

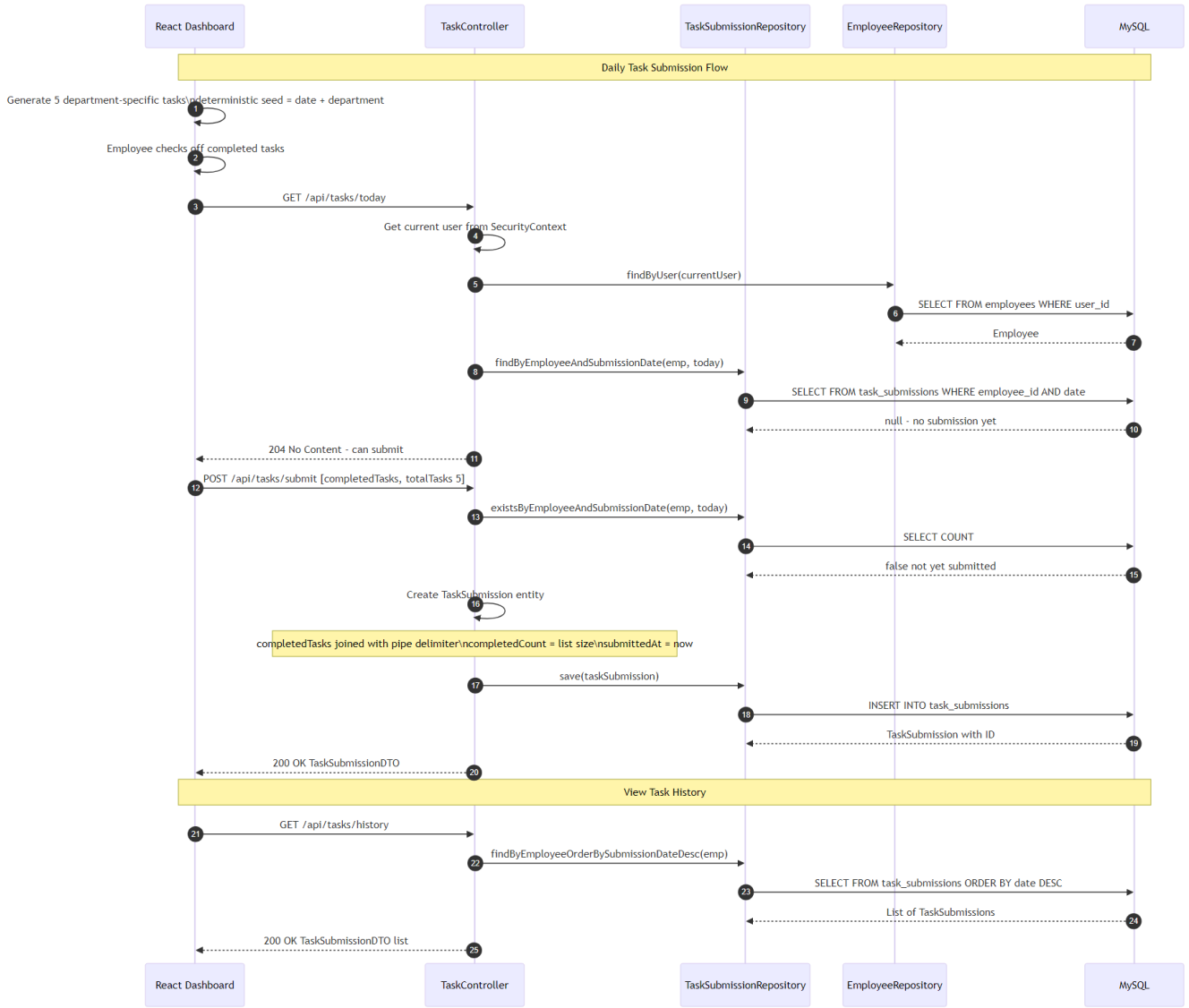
- ADMIN/HR can update all fields including salary and status
- Self-service (`/api/employees/me`) allows updating only `name` and `phone`
- Salary changes are restricted to ADMIN/HR roles

Delete Employee

- **ADMIN only** — HR cannot delete employees
- Cascading delete removes the linked User account
- Important: This invalidates the employee's login access immediately

8. Daily Task Submission Flow

The task tracking system provides department-specific daily checklists:



Task Generation Algorithm

Tasks are generated **client-side** using a deterministic algorithm:

1. Each department has **8 template tasks** (e.g., IT has "Review and merge pull requests", "Update system documentation")
2. A **seed** is computed from: `date string hash + department name hash`
3. Using a seeded shuffle, **5 tasks** are selected from the 8 templates
4. Each task gets a priority level: first = **High**, next two = **Medium**, last two = **Low**

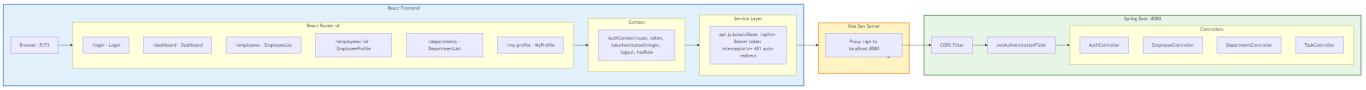
5. A fallback set of generic tasks exists for employees without a department

Submission Rules

- Each employee can submit **exactly once per day** (enforced by unique constraint on `employee_id + submissionDate`)
- The backend receives: list of completed task descriptions + total task count
- Tasks are stored as a `||`-delimited string in the database
- Submission history is accessible via the dashboard in reverse chronological order

9. Frontend-Backend Communication

The frontend communicates with the backend through a well-structured service layer:



Request Pipeline

1. **React Page** calls a service function (e.g., `employeeService.getAll()`)
2. **Service Layer** uses the shared `api.js` Axios instance
3. **Axios Request Interceptor** attaches `Authorization: Bearer <token>` from `localStorage`
4. **Vite Dev Proxy** forwards `/api/*` requests to `http://localhost:8080`
5. **Spring CORS Filter** validates the origin
6. **JWT Authentication Filter** validates the token and sets SecurityContext
7. **Controller** processes the request through the service/repository layers

Error Handling

- **401 Unauthorized**: Axios response interceptor clears `localStorage` token and redirects to `/login`
- **API errors**: Wrapped in `APIResponse<T>` with `success`, `message`, and `data` fields
- **Validation errors**: Backend returns descriptive error messages (e.g., "Employee code already exists")

10. API Endpoints Reference

Authentication (`/api/auth`)

Method	Endpoint	Auth	Request Body	Response
POST	`/api/auth/login`	Public	`{email, password}`	`{token, email, role, userId}`
GET	`/api/auth/me`	Authenticated	—	`User` entity

Employees (`/api/employees`)

Method	Endpoint	Auth	Request Body	Response
GET	`/api/employees`	ADMIN, HR	—	`APIResponse<List<EmployeeDTO>>`
GET	`/api/employees/{id}`	ADMIN, HR	—	`APIResponse<EmployeeDTO>`
POST	`/api/employees`	ADMIN, HR	`EmployeeRequest`	`APIResponse<EmployeeDTO>`
PUT	`/api/employees/{id}`	ADMIN, HR	`EmployeeRequest`	`APIResponse<EmployeeDTO>`
DELETE	`/api/employees/{id}`	ADMIN	—	`APIResponse<null>`
GET	`/api/employees/me`	Authenticated	—	`APIResponse<EmployeeDTO>`
PUT	`/api/employees/me`	Authenticated	`{name, phone}`	`APIResponse<EmployeeDTO>`

PUT	`/api/employees/me/change-password`	Authenticated	`ChangePasswordRequest`	`APIResponse<null>`

Departments (`/api/departments`)

Method	Endpoint	Auth	Request Body	Response
GET	`/api/departments`	ADMIN, HR	—	`APIResponse<List<DepartmentDTO>>`
GET	`/api/departments/{id}`	ADMIN, HR	—	`APIResponse<DepartmentDTO>`
POST	`/api/departments`	ADMIN, HR	`DepartmentDTO`	`APIResponse<DepartmentDTO>`
PUT	`/api/departments/{id}`	ADMIN, HR	`DepartmentDTO`	`APIResponse<DepartmentDTO>`
DELETE	`/api/departments/{id}`	ADMIN	—	`APIResponse<null>`

Tasks (`/api/tasks`)

Method	Endpoint	Auth	Request Body	Response
POST	`/api/tasks/submit`	Authenticated	`TaskSubmissionRequest`	`APIResponse<TaskSubmissionDTO>`
GET	`/api/tasks/today`	Authenticated	—	`TaskSubmissionDTO` or 204
GET	`/api/tasks/history`	Authenticated	—	`List<TaskSubmissionDTO>`

11. Database Schema

Tables Overview

```
┌──────────────────────────────────────────────────┐
│ users                                            │
├──────────────┬──────────────┬────────────────────┤
│ id (PK)      │ BIGINT       │ Auto-increment     │
│ email        │ VARCHAR(255) │ UNIQUE, NOT NULL   │
│ password     │ VARCHAR(255) │ NOT NULL (BCrypt)  │
│ role         │ ENUM         │ ADMIN, HR, EMPLOYEE│
│ jwt_secret   │ VARCHAR(512) │ Dynamic signing key│
└──────────────┴──────────────┴────────────────────┘

┌──────────────────────────────────────────────────┐
│ employees                                        │
├──────────────┬──────────────┬────────────────────┤
│ id (PK)      │ BIGINT       │ Auto-increment     │
│ emp_code     │ VARCHAR(255) │ UNIQUE, NOT NULL   │
│ name         │ VARCHAR(255) │ NOT NULL           │
│ email        │ VARCHAR(255) │ UNIQUE, NOT NULL   │
│ phone        │ VARCHAR(255) │                    │
│ designation  │ VARCHAR(255) │                    │
│ salary       │ DOUBLE       │                    │
│ status       │ ENUM         │ ACTIVE, INACTIVE   │
│ department_id│ BIGINT (FK)  │ → departments.id   │
│ user_id      │ BIGINT (FK)  │ → users.id (UNIQUE)│
└──────────────┴──────────────┴────────────────────┘

┌──────────────────────────────────────────────────┐
│ departments                                      │
```

```
├──────────────┼────────────────┼──────────────────────────────┤
│ id (PK)      │ BIGINT         │ Auto-increment               │
│ name         │ VARCHAR(255)   │ UNIQUE, NOT NULL             │
│ description  │ VARCHAR(255)   │                              │
└──────────────┴────────────────┴──────────────────────────────┘


┌──────────────────────────────────────────────────────────────┐
│ task_submissions                                             │
├──────────────────┼────────────────┼──────────────────────────┤
│ id (PK)          │ BIGINT         │ Auto-increment           │
│ employee_id (FK) │ BIGINT         │ → employees.id           │
│ submission_date  │ DATE           │ NOT NULL                 │
│ completed_tasks  │ TEXT           │ ||-delimited list        │
│ total_tasks      │ INT            │ NOT NULL                 │
│ completed_count  │ INT            │ NOT NULL                 │
│ submitted_at     │ DATETIME       │ NOT NULL                 │
└──────────────────┴────────────────┴──────────────────────────┘
  UNIQUE(employee_id, submission_date)
```

Initial Seed Data

Created by `DataInitializer.java` on first run:

Departments: IT, HR, Finance, Marketing, Operations

Users & Employees:

Email	Role	Employee Name	EmpCode	Department
admin@ems.com	ADMIN	—	—	—
hr@ems.com	HR	Sarah Johnson	EMP002	HR
employee@ems.com	EMPLOYEE	John Smith	EMP001	IT

All default passwords: `password123`

12. Security Architecture

Defense Layers

Layer	Mechanism	Implementation
Transport	HTTPS (production)	Configured at deployment level
Authentication	JWT Bearer tokens	`JwtAuthenticationFilter`
Authorization	Role-based method security	`@PreAuthorize` annotations
Password Storage	BCrypt hashing	`BCryptPasswordEncoder`
CORS	Origin whitelist	`SecurityConfig` CORS configuration
Session	Stateless (no server sessions)	`SessionCreationPolicy.STATELESS`
Token Invalidation	Dynamic per-user secrets	`User.jwtSecret` cleared on logout

JWT Security Model

```
Traditional JWT:     Static App Secret → Same key for all users
                     Compromise = ALL tokens compromised

EMS Dynamic JWT:     Per-User Secret → Unique 256-bit key per login
                     Compromise = Only ONE user's token compromised
                     Logout = Secret cleared → Token immediately invalid
```

CORS Configuration

Allowed origins:

- `http://localhost:5173` (Vite dev server)
- `http://localhost:3000` (alternative dev port)
- `http://localhost:3001` (alternative dev port)

13. Frontend Pages & Components

Page Structure

Page	Route	Role	Key Features
Login	`/login`	Public	Email/password form, error display, redirects on success
Dashboard	`/dashboard`	All	ADMIN/HR: stat cards (employee/department counts). EMPLOYEE: profile summary, daily task checklist, submission history
EmployeeList	`/employees`	ADMIN, HR	Searchable table, department filter chips with counts, Add/Edit/Delete buttons (Delete: ADMIN only)
EmployeeProfile	`/employees/:id`	ADMIN, HR	Create/Edit form, shows default password info on create, department dropdown
DepartmentList	`/departments`	ADMIN, HR	Table with inline Add/Edit dialog, Delete button (ADMIN only), employee count badge
MyProfile	`/my-profile`	EMPLOYEE	Read-only fields (empCode, email, department, designation), editable: name, phone, change password section

Component Hierarchy

```
<App>
  ├── <AuthProvider>          (Context provider)
  │   ├── <Login />           (Public route)
  │   └── <MainLayout>        (Protected wrapper)
  │       ├── <AppBar />      (Top bar: email, role, logout)
  │       ├── <Drawer />      (Side nav: role-filtered items)
  │       └── <Outlet />      (Renders active page)
  │           ├── <Dashboard />
  │           ├── <EmployeeList />
  │           ├── <EmployeeProfile />
  │           ├── <DepartmentList />
  │           └── <MyProfile />
  └── <Navigate />            (Redirect fallback)
```

14. Deployment Configuration

Docker Compose (MySQL)

```
services:
  mysql:
    image: mysql:8.0
    container_name: ems-mysql
    ports: 3306:3306
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: ems_db
      MYSQL_USER: ems_user
      MYSQL_PASSWORD: ems_password
    volumes: mysql_data (persistent)
    healthcheck: mysqladmin ping
```

Application Properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/ems_db?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=admin
spring.jpa.hibernate.ddl-auto=update
app.jwt.expiration=86400000   # 24 hours
```

Development Setup

1. Start MySQL: `docker-compose up -d`
2. Start Backend: `cd backend && mvn spring-boot:run` (port 8080)
3. Start Frontend: `cd frontend && npm install && npm run dev` (port 5173)
4. Access: `http://localhost:5173`

15. Demo Credentials

Role	Email	Password
ADMIN	admin@ems.com	password123
HR	hr@ems.com	password123
EMPLOYEE	employee@ems.com	password123
